

CSC 315 Week One TTTT2

Condensed Study Notes

Generated: 2026-05-24 22:56 UTC

CSC 315 : COMPUTER ARCHITECTURE AND ORGANIZATION I

Welcome to CSC 315: Computer Architecture and Organization I! This course will explore the fundamental principles behind how computers are built and how they work at a deep level.

Think of computer architecture as the blueprint for a computer, defining its components and how they interact. Organization, on the other hand, is about how those components are implemented and connected to make the architecture function.

In essence, we're going to learn what makes a computer tick, from its most basic parts to the overall design that allows it to process information.

Week One: Lesson One

This lesson serves as our initial dive into the world of Computer Architecture and Organization. Our focus will be on understanding the core principles that govern how computers are designed and how they operate.

Think of computer architecture as the blueprint of a house – it defines the overall structure, how rooms are connected, and the essential systems like plumbing and electricity. Computer organization, on the other hand, is like the actual building process, detailing how those systems are implemented and work together.

We will explore the fundamental concepts that make computers tick, laying the groundwork for understanding more complex systems. This foundational knowledge is crucial for anyone wanting to grasp the inner workings of the machines we use every day.

Introduction

Computer Architecture and Organization is about understanding the fundamental principles that guide how computers are designed and how they operate. Think of it like learning the blueprints and construction methods for a house.

This field focuses on a few key areas:

- **The Von Neumann Architecture:** This is a foundational model for how most modern computers are structured. It's characterized by a design where both program instructions and the data they operate on are stored in the same memory. This means the computer can treat instructions as data and vice-versa, which is a powerful concept.
- **Fundamental Building Blocks (Structure and Function):** We'll explore the basic components that make up a computer (its structure) and what each of these components does (its function). This is like understanding the different rooms, walls, and electrical wiring in a house, and knowing what each part is for.

1.1 Organization and Architecture

In computer design, a distinction is made between Computer Architecture and Computer Organization. While precise definitions can be tricky, there's a general agreement on what each covers.

Computer Architecture refers to the attributes of a system that are visible to a programmer. Think of it as the programmer's view of the computer – what they can directly interact with and what impacts how a program logically executes. A common term used interchangeably with computer architecture is **Instruction Set Architecture (ISA)**. The ISA defines things like:

- Instruction formats and opcodes (the codes that tell the computer what to do).
- The registers available to the programmer.
- How instructions and data are stored in memory.
- The effects of executed instructions on registers and memory.
- The algorithm for controlling instruction execution.

Examples of architectural attributes include the instruction set itself, the number of bits used for data types (like numbers or characters), how input/output (I/O) is handled, and how memory is addressed.

Computer Organization, on the other hand, refers to the operational units of a computer and their interconnections that realize the architectural specifications. This is about *how* the architectural features are implemented in hardware. Organizational decisions are often based on factors like:

- How frequently a specific instruction (like multiplication) is expected to be used.
- The speed of different implementation approaches.
- The cost and physical size of specialized hardware units.

For instance, it's an architectural decision whether a computer *will have* a multiply instruction. It's an organizational decision *how* that instruction will be performed – perhaps by a dedicated multiply unit or by repeatedly using the add unit. Organizational attributes are hardware details that are usually transparent to the programmer, such as control signals, interfaces with peripherals, and the type of memory technology used.

A computer's organization must be designed to implement its architecture. Therefore, understanding organization requires a good grasp of architecture.

Historically, this distinction has been important. Many manufacturers offer computer families with the same architecture but different organizations. This allows for compatibility at the architectural level while offering different performance and cost points through varied organization. The IBM System/370 is a classic example, maintaining the same architecture for decades while its organization evolved.

1.2 Structure and Function

To understand a complex system like a computer, we can approach its description in two ways: building up from the smallest parts or breaking down from a high-level view. Research suggests that starting with a top-down approach, and decomposing the system into its subparts, is the clearest way to explain it.

Computers, like many complex systems, are built hierarchically. This means they are made up of interrelated subsystems, and each of those subsystems is itself made up of smaller subsystems, continuing down to the most basic components.

At every level of this hierarchy, designers focus on two key aspects:

- **Structure:** This refers to how the different components of a system are connected and interrelated. Think of it like the blueprint of a house, showing where the rooms, walls, and doors are placed.
- **Function:** This describes what each individual component does and how it operates as part of the overall structure. In our house analogy, function would be what each room is used for (e.g., the kitchen is for cooking, the bedroom is for sleeping).

Structure (The Internal structure of the Computer)

The internal structure of a computer can be broken down into several key components that work together.

At the highest level, a traditional computer has four main structural components:

- **Central Processing Unit (CPU):** Often called the processor, the CPU is the brain of the computer. It controls the computer's operations and performs its data processing functions.
- **Main memory:** This is where the computer stores data.
- **I/O (Input/Output):** This component handles the movement of data between the computer and the outside world (e.g., keyboard, screen).
- **System interconnection:** This is the communication pathway that allows the CPU, main memory, and I/O to talk to each other. A common example is a **system bus**, which is a set of wires connecting these components.

The CPU itself is a complex component with its own major structural parts:

- **Arithmetic and logic unit (ALU):** This is where the actual number crunching and logical operations happen.
- **Control unit:** This directs the operations of the CPU and, by extension, the entire computer. One way to implement a control unit is through **microprogramming**, where it executes **microinstructions** that define its functionality.
- **Registers:** These are small, fast storage locations located inside the CPU, used for temporary data storage during processing.
- **CPU interconnection:** This is the mechanism that allows the control unit, ALU, and registers within the CPU to communicate with each other.

Traditionally, computers had a single processor. However, modern computers increasingly use multiple processors. Techniques like **parallelism** and **pipelining** are used to enhance performance, adding complexity to the computer's organization.

Function

Computers, at their core, perform four fundamental functions. These functions work together to allow a computer to process information and interact with its environment.

1. **Data processing:** This involves manipulating data in various ways. Examples include simple operations like counting, swapping data between locations, performing mathematical additions, and comparing data values to make decisions.

2. **Control:** A dedicated unit within the computer, the control unit, acts like a conductor of an orchestra. It manages all the computer's resources and directs the operation of other functional parts based on the instructions it receives.

3. **Data storage:** Computers need to store data both temporarily and for the long term.

- **Short-term storage** is essential for holding the data that the computer is actively working on at any given moment. Think of it like keeping the ingredients you're currently using on your kitchen counter.
- **Long-term storage** is used to keep files and data for later retrieval and modification, much like storing your recipes in a cookbook for future use.

4. **Data movement:** This function involves transferring data into and out of the computer.

- **Input/Output (I/O)** refers to the process of receiving data from or sending data to devices directly connected to the computer. These devices are called peripherals (e.g., a keyboard, monitor, or printer).
- **Data communications** involves moving data over longer distances, to or from devices that are not directly connected.

multicore computer structure

Many modern computers are built around a **multicore processor**. This is a single chip, often called a **processor**, that contains multiple individual processing units called **cores**. Think of a core as a mini-CPU, capable of fetching, decoding, and executing instructions on its own. Some cores might be general-purpose, similar to a CPU in an older, single-core system, while others could be specialized for specific tasks like complex mathematical operations.

These multicore processors are typically housed on the **motherboard**, which is the main printed circuit board in a computer. The motherboard acts like a central hub, connecting the processor chip to other essential components. It provides slots for the processor, memory chips, and various input/output (I/O) controller chips. The motherboard itself is a sophisticated piece of engineering, made of layers with copper pathways that interconnect all the different electronic components.

While the processor chip contains the cores and often an **L3 cache** (a fast memory accessible by all cores), it also requires logic to manage communication between these cores, the cache, and the rest of the motherboard circuitry. This complex interplay allows multiple cores to work together, significantly enhancing the computer's processing power.

Inside a Single Processor Core

A single processor core, which is a part of the larger processor chip, contains several key functional elements.

Core Functional Elements:

- **Instruction Logic:** This handles fetching instructions from memory and then decoding them. Decoding means figuring out what operation the instruction wants the processor to perform and where to find the necessary data (operands) or where to store the results.
- **Arithmetic and Logic Unit (ALU):** This is where the actual calculations and logical comparisons happen, based on the decoded instructions.
- **Load/Store Logic:** This manages the movement of data between the core and main memory, typically using a cache. Think of it as the core's dedicated delivery service for data.
- **Cache Memory (L1 and L2):** Cores have their own small, fast memory called cache to speed up access to frequently used instructions and data. This is often split into an **instruction cache (I-cache)** for instructions and a **data cache** for data. Some cores also include an **L2 cache** as part of the core itself, which can also be split or combined.

While this describes the general layout, the exact physical arrangement of these elements can vary between different processor designs.

1.3 Von Neumann architecture

The Von Neumann architecture, described by mathematician John Von Neumann in the 1940s, outlines the fundamental arrangement of a computer system. It's a model that most modern computers still follow.

A Von Neumann-based computer is characterized by the following:

- **Serial Instruction Execution:** It executes programs by processing instructions one after another in a sequential manner. This is achieved through a **fetch-decode-execute cycle**, where the computer fetches an instruction, figures out what it means (decodes it), and then carries it out (executes it).
- **Unified Memory for Instructions and Data:** A key feature is that it uses a single memory space and a single path to access both program instructions and the data those instructions operate on. The computer doesn't inherently know if a bit pattern in memory is an instruction or data; it relies on the location and context to interpret it correctly.
- **Single Processor:** Traditionally, this architecture involves a single processor.

When we discuss the Von Neumann architecture, we are primarily referring to the relationship between the hardware components that make up such a computer.

Von Neumann bottleneck

The Von Neumann architecture, while foundational, has a limitation known as the 'Von Neumann bottleneck'. This bottleneck arises because instructions can only be processed one at a time and must be executed sequentially.

Even with improvements like more RAM, faster cache memory, or quicker individual components, significant gains in CPU performance are hindered by this inherent sequential nature. To truly boost CPU efficiency, a fundamental re-evaluation of the CPU's design itself is necessary to overcome these limitations.